

Updates with Aggregation Pipeline

To perform update operations, you can use the aggregation pipeline. You can build and execute aggregation pipelines to perform updates in MongoDB Atlas, MongoDB Compass, MongoDB Shell, or Drivers.

With the update operations, the aggregation pipeline can consist of the following stages:

- `$addField`
- `$set`
- `$project`
- `$unset`
- `$replaceRoot`
- `$replaceWith`

Using the aggregation pipeline allows for a more expressive update statement, such as expressing conditional updates based on current field values or updating one field using the value of another field(s).

Create an Update Aggregation Pipeline in Atlas

You can use the MongoDB Atlas UI to build an aggregation pipeline to perform updates. To create and execute aggregation pipelines in the MongoDB Atlas UI, you must have the `Project Data Access Read Only` role or higher.

1 Access the Aggregation Pipeline Builder.

1 Select the database for the collection.

The main panel and **Namespaces** on the left side list the collections in the database.

2 Select the collection.

Select the collection on the left-hand side or in the main panel. The main panel displays the **Find**, **Indexes**, and **Aggregation** views.

3 Select the Aggregation view.

When you first open the **Aggregation** view, Atlas displays an empty aggregation pipeline.

2 Create an aggregation pipeline to perform updates.

1 Select an aggregation stage.

Select an aggregation stage from the **Select** drop-down menu in the bottom-left panel.

The toggle to the right of the drop-down menu dictates whether the stage is enabled.

To perform updates with an aggregation, use one of these stages:

- `$addField`
- `$set`
- `$project`
- `$unset`
- `$replaceRoot`
- `$replaceWith`

2 Fill in your aggregation stage.

Fill in your stage with the appropriate values. If Comment Mode is enabled, the pipeline builder provides syntactic guidelines for your selected stage.

As you modify your stage, Atlas updates the preview documents on the right based on the results of the current stage.

For examples of what you might include in your aggregation stage, see the examples on this page.

Add stages as needed. For more information on creating aggregation pipelines in Atlas, refer to [Create an Aggregation Pipeline](#).

3 Export the aggregation pipeline.

1 Click Export to Language.

You can find this button at the top of the pipeline builder.

2

Select your desired export language.

In the **Export Pipeline To** menu, select your desired language.

The **My Pipeline** pane on the left displays your pipeline in MongoDB Shell syntax. You can copy this directly to execute your pipeline in the MongoDB Shell.

The pane on the right displays your pipeline in the selected language. Select your preferred language.

3

Select options, if desired.

(Optional): Check the **Include Import Statements** option to include the required import statements for the language selected.

(Optional): Check the **Include Driver Syntax** option to include Driver-specific code to:

- Initialize the client
- Specify the database and collection
- Perform the aggregation operation

4

Copy the pipeline.

Click the **Copy** button at the top-right of the pipeline to copy the pipeline for the selected language to your clipboard. Paste the copied pipeline into your application.

Examples

The following examples demonstrate how to use the aggregation pipeline stages `$set`, `$replaceRoot`, and `$addFields` to perform updates.

updateOne with \$set

Create an example `students` collection (if the collection does not currently exist, insert operations will create the collection):

```
db.students.insertMany( [
  { _id: 1, test1: 95, test2: 92, test3: 90, modified: new
    Date("01/05/2020") },
  { _id: 2, test1: 98, test2: 100, test3: 102, modified: new
    Date("01/05/2020") },
  { _id: 3, test1: 95, test2: 110, modified: new Date("01/04/2020") }
] )
```

To verify, query the collection:

```
db.students.find()
```

The following `db.collection.updateOne()` operation uses an aggregation pipeline to update the document with `_id: 3`:

```
db.students.updateOne( { _id: 3 }, [ { $set: { "test3": 98, modified:
  "$$NOW" } } ] )
```

Specifically, the pipeline consists of a `$set` stage which adds the `test3` field (and sets its value to 98) to the document and sets the `modified` field to the current datetime. The operation uses the aggregation variable `NOW` for the current datetime. To access the variable, prefix with `$$` and enclose in quotes.

To verify the update, you can query the collection:

```
db.students.find().pretty()
```

updateMany with \$replaceRoot and \$set

Create an example `students2` collection (if the collection does not currently exist, insert operations will create the collection):

```
db.students2.insertMany( [
  { "_id" : 1, quiz1: 8, test2: 100, quiz2: 9, modified: new
    Date("01/05/2020") },
  { "_id" : 2, quiz2: 5, test1: 80, test2: 89, modified: new
    Date("01/05/2020") },
  ] )
```

To verify, query the collection:

```
db.students2.find()
```

The following `db.collection.updateMany()` operation uses an aggregation pipeline to standardize the fields for the documents (i.e. documents in the collection should have the same fields) and update the `modified` field:

```
db.students2.updateMany( {},
[
  { $replaceRoot: { newRoot:
  { $mergeObjects: [ { quiz1: 0, quiz2: 0, test1: 0, test2: 0 }, "$$ROOT"
  ] } }
},
{ $set: { modified: "$$NOW" } }
]
)
```

Specifically, the pipeline consists of:

- a `$replaceRoot` stage with a `$mergeObjects` expression to set default values for the `quiz1`, `quiz2`, `test1` and `test2` fields. The aggregation variable `ROOT` refers to the current document being modified. To access the variable, prefix with `$$` and enclose in quotes. The current document fields will override the default values.
- a `$set` stage to update the `modified` field to the current datetime. The operation uses the aggregation variable `NOW` for the current datetime. To access the variable, prefix with `$$` and enclose in quotes.

To verify the update, you can query the collection:

```
db.students2.find()
```

updateMany with \$set

Create an example `students3` collection (if the collection does not currently exist, insert operations will create the collection):

```
db.students3.insertMany( [
  { "_id" : 1, "tests" : [ 95, 92, 90 ], "modified" : ISODate("2019-01-01T00:00:00Z") },
  { "_id" : 2, "tests" : [ 94, 88, 90 ], "modified" : ISODate("2019-01-01T00:00:00Z") },
  { "_id" : 3, "tests" : [ 70, 75, 82 ], "modified" : ISODate("2019-01-01T00:00:00Z") }
] );
```

To verify, query the collection:

```
db.students3.find()
```

The following `db.collection.updateMany()` operation uses an aggregation pipeline to update the documents with the calculated grade average and letter grade.

```
db.students3.updateMany(
{ },
[
{ $set: { average : { $trunc: [ { $avg: "$tests" }, 0 ] }, modified:
"$NOW" } },
{ $set: { grade: { $switch: {
branches: [
{ case: { $gte: [ "$average", 90 ] }, then: "A" },
{ case: { $gte: [ "$average", 80 ] }, then: "B" },
{ case: { $gte: [ "$average", 70 ] }, then: "C" },
{ case: { $gte: [ "$average", 60 ] }, then: "D" }
},
default: "F"
} } } }
]
)
```

Specifically, the pipeline consists of:

- a `$set` stage to calculate the truncated average value of the `tests` array elements and to update the `modified` field to the current datetime. To calculate the truncated average, the stage uses the `$avg` and `$trunc` expressions. The operation uses the aggregation variable `NOW` for the current datetime. To access the variable, prefix with `$$` and enclose in quotes.
- a `$set` stage to add the `grade` field based on the `average` using the `$switch` expression.

To verify the update, you can query the collection:

```
db.students3.find()
```

updateOne with \$set

Create an example `students4` collection (if the collection does not currently exist, insert operations will create the collection):

```
db.students4.insertMany( [
{ "_id" : 1, "quizzes" : [ 4, 6, 7 ] },
{ "_id" : 2, "quizzes" : [ 5 ] },
{ "_id" : 3, "quizzes" : [ 10, 10, 10 ] }
] )
```

To verify, query the collection:

```
db.students4.find()
```

The following `db.collection.updateOne()` operation uses an aggregation pipeline to add quiz scores to the document with `_id: 2`:

```
db.students4.updateOne( { _id: 2 },  
[ { $set: { quizzes: { $concatArrays: [ "$quizzes", [ 8, 6 ] ] } } } ]  
)
```

To verify the update, query the collection:

```
db.students4.find()
```

updateMany with \$addFields

Create an example `temperatures` collection that contains temperatures in Celsius (if the collection does not currently exist, insert operations will create the collection):

```
db.temperatures.insertMany( [  
{ "_id" : 1, "date" : ISODate("2019-06-23"), "tempsC" : [ 4, 12, 17 ] },  
{ "_id" : 2, "date" : ISODate("2019-07-07"), "tempsC" : [ 14, 24, 11 ]  
},  
{ "_id" : 3, "date" : ISODate("2019-10-30"), "tempsC" : [ 18, 6, 8 ] }  
)
```

To verify, query the collection:

```
db.temperatures.find()
```

The following `db.collection.updateMany()` operation uses an aggregation pipeline to update the documents with the corresponding temperatures in Fahrenheit:

```
db.temperatures.updateMany( { },  
[  
{ $addFields: { "tempsF": {  
$map: {  
input: "$tempsC",  
as: "celsius",  
in: { $add: [ { $multiply: [ "$$celsius", 9/5 ] }, 32 ] }  
}  
} } }  
]  
)
```

Specifically, the pipeline consists of an `$addFields` stage to add a new array field `tempsF` that contains the temperatures in Fahrenheit. To convert each celsius temperature in the `tempsC` array to Fahrenheit, the stage uses the `$map` expression with `$add` and `$multiply` expressions.

To verify the update, you can query the collection:

```
db.temperatures.find()
```



Update with let Variables

New in version 5.0.

To define variables that you can access elsewhere in the command, use the `let` option.

NOTE

To filter results using a variable, you must access the variable within the `$expr` operator.

Create a collection `cakeFlavors`:

```
db.cakeFlavors.insertMany( [
  { _id: 1, flavor: "chocolate" },
  { _id: 2, flavor: "strawberry" },
  { _id: 3, flavor: "cherry" }
] )
```



The following `updateOne` command uses variables set with the `let` option:

- The `targetFlavor` variable is set to `cherry`. This variable is used in the `$eq` expression to specify the match filter.
- The `newFlavor` variable is set to `orange`. This variable is used in the `$set` operator to specify the updated `flavor` value for the matched document.

```
db.cakeFlavors.updateOne(
{
  $expr: { $eq: [ "$flavor", "$$targetFlavor" ] }
},
[
  {
    $set: { flavor: "$$newFlavor" }
  }
],
{
  let: { targetFlavor: "cherry", newFlavor: "orange" }
}
)
```



After you run the preceding update operation, the `cakeFlavors` collection contains these documents:

```
[
  { _id: 1, flavor: 'chocolate' },
  { _id: 2, flavor: 'strawberry' },
  { _id: 3, flavor: 'orange' }
]
```

Additional Examples

See also the various update method pages for additional examples:

- `db.collection.updateOne`
- `db.collection.updateMany`
- `db.collection.findOneAndUpdate()`
- `db.collection.findAndModify()`
- `Bulk.find.update()`
- `Bulk.find.updateOne()`
- `Bulk.find.upsert()`

